



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch54 并发控制

Software Institute
Nanjing University
Bei Jia

事务并发

- 多用户数据库系统，允许多个用户同时使用的数据库系统
 - 飞机定票数据库系统
 - 银行数据库系统
- 特点：在同一时刻并发运行的事务数可达数百上千个
- 事务并发执行带来的问题
 - 会产生多个事务同时存取同一数据的情况
 - 可能会存取和存储不正确的数据，破坏事务隔离性和数据库的一致性
- 数据库管理系统必须提供并发控制机制，并发控制机制是衡量一个数据库管理系统性能的重要标志之一



多事务执行方式 1

● 事务串行执行

- 每个时刻只有一个事务运行，其他事务必须等到这个事务结束以后方能运行
- 不能充分利用系统资源，发挥数据库共享资源的特点

T₁

T₂

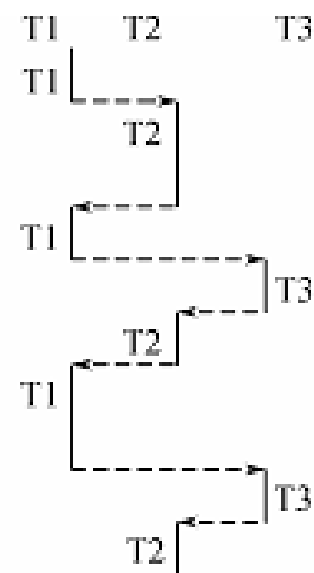
T₃

事务的串行执行方式



多事务执行方式 2

- 交叉并发方式 (Interleaved Concurrency)
 - 在单处理机系统中，事务的并行执行是这些并行事务的并行操作轮流交叉运行
 - 单处理机系统中的并行事务并没有真正地并行运行，但能够减少处理机的空闲时间，提高系统的效率



(b) 事务的交叉并发执行方式

多事务执行方式 3

- 同时并发方式 (simultaneous concurrency)
 - 多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并行运行
 - 最理想的并发方式，但受制于硬件环境
 - 更复杂的并发方式机制



并发控制

- 事务是并发控制的基本单位
- 并发控制机制的任务
 - 对并发操作进行正确调度
 - 保证事务的隔离性
 - 保证数据库的一致性



不一致性的例子 1

- [例11.1]飞机订票系统中的一个活动序列

- ① 甲售票点(事务 T_1)读出某航班的机票余额 A , 设 $A=16$;
 - ② 乙售票点(事务 T_2)读出同一航班的机票余额 A , 也为16;
 - ③ 甲售票点卖出一张机票, 修改余额 $A \leftarrow A-1$, 所以 A 为15, 把 A 写回数据库;
 - ④ 乙售票点也卖出一张机票, 修改余额 $A \leftarrow A-1$, 所以 A 为15, 把 A 写回数据库
- 结果明明卖出两张机票, 数据库中机票余额只减少1



不一致性的例子 2

- 这种情况称为数据库的不一致性，是由并发操作引起的。
- 在并发操作情况下，对 T_1 、 T_2 两个事务的操作序列的调度是随机的。
- 若按上面的调度序列执行， T_1 事务的修改就被丢失。
 - 原因：第4步中 T_2 事务修改A并写回后覆盖了 T_1 事务的修改



并发操作带来的数据不一致性

- 丢失修改 (Lost Update)
- 不可重复读 (Non-repeatable Read)
- 读“脏”数据 (Dirty Read)
- 记号
 - $R(x)$: 读数据 x
 - $W(x)$: 写数据 x



丢失修改

- 两个事务 T_1 和 T_2 读入同一数据并修改, T_2 的提交结果破坏了 T_1 提交的结果, 导致 T_1 的修改被丢失。
- 上面飞机订票例子就属此类

T_1	T_2
① $R(A)=16$	
②	$R(A)=16$
③ $A \leftarrow A-1$ $W(A)=15$	
④	$A \leftarrow A-1$ $W(A)=15$



不可重复读 1

- 不可重复读是指事务 T_1 读取数据后，事务 T_2 执行更新操作，使 T_1 无法再现前一次读取结果。
- 不可重复读包括三种情况，后两种不可重复读有时也称为幻影现象（Phantom Row）：
 - 事务 T_1 读取某一数据后，事务 T_2 对其做了修改，当事务 T_1 再次读该数据时，得到与前一次不同的值
 - 事务 T_1 按一定条件从数据库中读取了某些数据记录后，事务 T_2 删除了其中部分记录，当 T_1 再次按相同条件读取数据时，发现某些记录神秘地消失了。
 - 事务 T_1 按一定条件从数据库中读取某些数据记录后，事务 T_2 插入了一些记录，当 T_1 再次按相同条件读取数据时，发现多了一些记录。



不可重复读 2

T_1	T_2
① $R(A)=50$	
$R(B)=100$	
求和=150	
②	$R(B)=100$
	$B \leftarrow B * 2$
	$W(B)=200$
③ $R(A)=50$	
$R(B)=200$	
求和=250	
(验算不对)	

- T_1 读取 $B=100$ 进行运算
- T_2 读取同一数据 B , 对其进行修改后将 $B=200$ 写回数据库。
- T_1 为了对读取值校对重读 B , B 已为200, 与第一次读取值不一致



读“脏”数据 1

- 读“脏”数据是指：
 - 事务 T_1 修改某一数据，并将其写回磁盘
 - 事务 T_2 读取同一数据后， T_1 由于某种原因被撤销
 - 这时 T_1 已修改过的数据恢复原值， T_2 读到的数据就与数据库中的数据不一致
 - T_2 读到的数据就为“脏”数据，即不正确的数据



读“脏”数据 2

T_1	T_2
① $R(C)=100$	
$C \leftarrow C * 2$	
$W(C)=200$	
②	$R(C)=200$
③ ROLLBACK	
C恢复为100	

- T_1 将C值修改为200, T_2 读到C为200
- T_1 由于某种原因撤销, 其修改作废, C恢复原值100
- 这时 T_2 读到的C为200, 与数据库内容不一致, 就是“脏”数据



数据不一致性及并发控制

- 数据不一致性：由于并发操作破坏了事务的隔离性
- 并发控制就是要用正确的方式调度并发操作，使一个用户事务的执行不受其他事务的干扰，从而避免造成数据的不一致性
- 对数据库的应用有时允许某些不一致性，例如有些统计工作涉及数据量很大，读到一些“脏”数据对统计精度没什么影响，可以降低对一致性的要求以减少系统开销



并发控制的主要技术

- 并发控制的主要技术
 - 封锁(Locking)
 - 时间戳(Timestamp)
 - 乐观控制法
 - 多版本并发控制(MVCC)





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch55 封锁

Software Institute
Nanjing University
Bei Jia

封锁

- 封锁就是事务T在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁
- 加锁后事务T就对该数据对象有了一定的控制，在事务T释放它的锁之前，其它的事务不能更新此数据对象。
- 封锁是实现并发控制的一个非常重要的技术
- 一个事务对某个数据对象加锁后究竟拥有什么样的控制由封锁的类型决定。
- 基本封锁类型
 - 排它锁（Exclusive Locks，简记为X锁）
 - 共享锁（Share Locks，简记为S锁）



排它锁与共享锁

● 排它锁又称为写锁

- 若事务T对数据对象A加上X锁，则只允许T读取和修改A，其它任何事务都不能再对A加任何类型的锁，直到T释放A上的锁
- 保证其他事务在T释放A上的锁之前不能再读取和修改A

● 共享锁又称为读锁

- 若事务T对数据对象A加上S锁，则事务T可以读A但不能修改A，其它事务只能再对A加S锁，而不能加X锁，直到T释放A上的S锁
- 保证其他事务可以读A，但在T释放A上的S锁之前不能对A做任何修改



锁的相容矩阵 1

$T_2 \backslash T_1$	X	S	—
X	N	N	Y
S	N	Y	Y
—	Y	Y	Y

Y=Yes, 相容的请求
N=No, 不相容的请求



锁的相容矩阵 2

- 在锁的相容矩阵中：
- 最左边一列表示事务 T_1 已经获得的数据对象上的锁的类型，其中横线表示没有加锁。
- 最上面一行表示另一事务 T_2 对同一数据对象发出的封锁请求。
- T_2 的封锁请求能否被满足用矩阵中的Y和N表示
 - Y表示事务 T_2 的封锁要求与 T_1 已持有的锁相容，封锁请求可以满足
 - N表示 T_2 的封锁请求与 T_1 已持有的锁冲突， T_2 的请求被拒绝



封锁协议

- 在运用X锁和S锁对数据对象加锁时，需要约定一些规则，这些规则为封锁协议（Locking Protocol）。
 - 何时申请X锁或S锁
 - 持锁时间
 - 何时释放
- 对封锁方式规定不同的规则，就形成了各种不同的封锁协议，它们分别在不同的程度上为并发操作的正确调度提供一定的保证。



一级封锁协议 1

- 一级封锁协议
 - 事务T在修改数据R之前必须先对其加X锁，直到事务结束才释放。
 - ◆ 正常结束 (COMMIT)
 - ◆ 非正常结束 (ROLLBACK)
- 一级封锁协议可防止丢失修改，并保证事务T是可恢复的。
- 在一级封锁协议中，如果仅仅是读数据不对其进行修改，是不需要加锁的，所以它不能保证可重复读和不读“脏”数据。



一级封锁协议 1

T_1	T_2
① Xlock A	
② $R(A)=16$	
	Xlock A
③ $A \leftarrow A-1$	等待
$W(A)=15$	等待
Commit	等待
Unlock A	等待
④	获得Xlock A
	$R(A)=15$
	$A \leftarrow A-1$
⑤	$W(A)=14$
	Commit
	Unlock A

- 事务 T_1 在读A进行修改之前先对A加X锁
- 当 T_2 再请求对A加X锁时被拒绝
- T_2 只能等待 T_1 释放A上的锁后获得对A的X锁
- 这时 T_2 读到的A已经是 T_1 更新过的值15
- T_2 按此新的A值进行运算，并将结果值 $A=14$ 写回到磁盘。避免了丢失 T_1 的更新。



二级封锁协议 1

- 二级封锁协议
 - 一级封锁协议加上事务T在读取数据R之前必须先对其加S锁，读完后即可释放S锁。
- 二级封锁协议可以防止丢失修改和读“脏”数据。
- 在二级封锁协议中，由于读完数据后即可释放S锁，所以它不能保证可重复读。



二级封锁协议 2

T_1	T_2
① Xlock C	
R(C)=100	
$C \leftarrow C * 2$	
W(C)=200	
②	Slock C
	等待
③ROLLBACK	等待
(C恢复为100)	等待
Unlock C	等待
④	获得Slock C
	R(C)=100
⑤	Commit C
	Unlock C

- 事务 T_1 在对C进行修改之前, 先对C加X锁, 修改其值后写回磁盘
- T_2 请求在C上加S锁, 因 T_1 已在C上加了X锁, T_2 只能等待
- T_1 因某种原因被撤销, C恢复为原值100
- T_1 释放C上的X锁后 T_2 获得C上的S锁, 读C=100。避免了 T_2 读“脏”数据



三级封锁协议 1

- 三级封锁协议
 - 一级封锁协议加上事务T在读取数据R之前必须先对其加S锁，直到事务结束才释放。
- 三级封锁协议可防止丢失修改、读脏数据和不可重复读。



三级封锁协议 1

T ₁	T ₂
① Slock A	
Slock B	
R(A)=50	
R(B)=100	
求和=150	
②	Xlock B
	等待
③ R(A)=50	等待
R(B)=100	等待
求和=150	等待
Commit	等待
Unlock A	等待
Unlock B	等待
④	获得XlockB
	R(B)=100
	B ← B*2
⑤	W(B)=200
	Commit
	Unlock B

- 事务T₁在读A, B之前, 先对A, B加S锁
- 其他事务只能再对A, B加S锁, 而不能加X锁, 即其他事务只能读A, B, 而不能修改
- 当T₂为修改B而申请对B的X锁时被拒绝只能等待T₁释放B上的锁
- T₁为验算再读A, B, 这时读出的B仍是100, 求和结果仍为150, 即可重复读
- T₁结束才释放A, B上的S锁。T₂才获得对B的X锁



封锁协议小结

- 三级协议的主要区别
 - 什么操作需要申请封锁以及何时释放锁（即持锁时间）
- 不同的封锁协议使事务达到的一致性级别不同
 - 封锁协议级别越高，一致性程度越高

	X锁		S锁		一致性保证		
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不读“脏”数据	可重复读
一级封锁协议		√			√		
二级封锁协议		√	√		√	√	
三级封锁协议		√		√	√	√	√



活锁 1

- 事务 T_1 封锁了数据R
- 事务 T_2 又请求封锁R, 于是 T_2 等待。
- T_3 也请求封锁R, 当 T_1 释放了R上的封锁之后系统首先批准了 T_3 的请求, T_2 仍然等待。
- T_4 又请求封锁R, 当 T_3 释放了R上的封锁之后系统又批准了 T_4 的请求.....
- T_2 有可能永远等待, 这就是活锁的情形
- 避免活锁: 采用先来先服务的策略
 - 当多个事务请求封锁同一数据对象时
 - 按请求封锁的先后次序对这些事务排队
 - 该数据对象上的锁一旦释放, 首先批准申请队列中第一个事务获得锁



活锁 2

T_1	T_2	T_3	T_4
Lock R	•	•	•
	•	•	•
•		•	
•	Lock R		
•	等待	Lock R	
Unlock R	等待	等待	Lock R
	等待	等待	等待
	等待	Lock R	等待
•	等待	•	等待
•	等待	Unlock	等待
•	等待	•	Lock R
	等待	•	•
			•

(a)活 锁



死锁 1

- 事务 T_1 封锁了数据 R_1
- T_2 封锁了数据 R_2
- T_1 又请求封锁 R_2 ，因 T_2 已封锁了 R_2 ，于是 T_1 等待 T_2 释放 R_2 上的锁
- 接着 T_2 又申请封锁 R_1 ，因 T_1 已封锁了 R_1 ， T_2 也只能等待 T_1 释放 R_1 上的锁
- 这样 T_1 在等待 T_2 ，而 T_2 又在等待 T_1 ， T_1 和 T_2 两个事务永远不能结束，形成死锁



死锁 2

T_1	T_2
Lock R_1	•
	•
	•
•	Lock R_2
•	•
•	•
Lock R_2	•
等待	
等待	
等待	Lock R_1
等待	等待
等待	等待
	•
	•
	•

(b) 死锁



死锁的预防

- 产生死锁的原因是两个或多个事务都已封锁了一些数据对象，然后又都请求对已为其他事务封锁的数据对象加锁，从而出现死等待。
- 预防死锁的发生就是要破坏产生死锁的条件
- 一次封锁法，要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行
- 存在的问题
 - 降低系统并发度
 - 难于事先精确确定封锁对象
 - ◆ 数据库中数据是不断变化的，原来不要求封锁的数据，在执行过程中可能会变成封锁对象，所以很难事先精确地确定每个事务所要封锁的数据对象。
 - ◆ 解决方法：将事务在执行过程中可能要封锁的数据对象全部加锁，这就进一步降低了并发度。
- 顺序封锁法，预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁。
- 存在的问题
 - 维护成本
 - ◆ 数据库系统中封锁的数据对象极多，并且随数据的插入、删除等操作而不断地变化，要维护这样的资源的封锁顺序非常困难，成本很高。
 - 难以实现
 - ◆ 事务的封锁请求可以随着事务的执行而动态地决定，很难事先确定每一个事务要封锁哪些对象，因此也就很难按规定的顺序去施加封锁

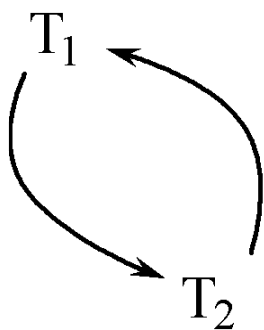


死锁的诊断

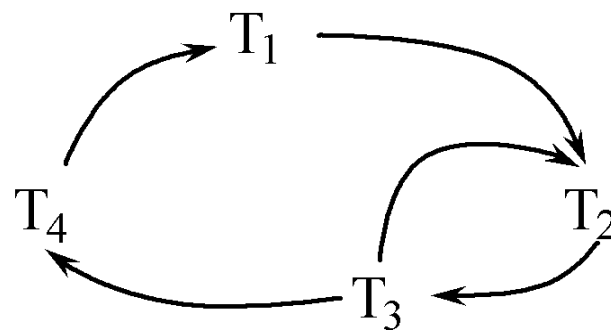
- 数据库管理系统在解决死锁的问题上更普遍采用的是诊断并解除死锁的方法
 - 在操作系统中广为采用的预防死锁的策略并不太适合数据库的特点
- 超时法，如果一个事务的等待时间超过了规定的时限，就认为发生了死锁
- 优点
 - 实现简单
- 缺点
 - 有可能误判死锁
 - 时限若设置得太长，死锁发生后不能及时发现
- 等待图法，并发控制子系统周期性地（比如每隔数秒）生成事务等待图，检测事务。如果发现图中存在回路，则表示系统中出现了死锁。
 - 事务等待图是一个有向图 $G=(T, U)$
 - T 为结点的集合，每个结点表示正运行的事务
 - U 为边的集合，每条边表示事务等待的情况
 - 若 T_1 等待 T_2 ，则 T_1 ， T_2 之间划一条有向边，从 T_1 指向 T_2



等待图法



(a)



(b)

- 图(a)中, 事务 T_1 等待 T_2 , T_2 等待 T_1 , 产生了死锁
- 图(b)中, 事务 T_1 等待 T_2 , T_2 等待 T_3 , T_3 等待 T_4 , T_4 又等待 T_1 , 产生了死锁
- 图(b)中, 事务 T_3 可能还等待 T_2 , 在大回路中又有小的回路

死锁的解除

- 解除死锁

- 选择一个处理死锁代价最小的事务，将其撤消
- 释放此事务持有的所有的锁，使其它事务能继续运行下去





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch56 事务调度

Software Institute
Nanjing University
Bei Jia

并发调度的可串行性

- 数据库管理系统对并发事务不同的调度可能会产生不同的结果
- 串行调度是正确的
- 执行结果等价于串行调度的调度也是正确的，称为可串行化调度



可串行化调度

- 可串行化(Serializable)调度
 - 多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同
- 可串行性(Serializability)
 - 是并发事务正确调度的准则
 - 一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度



可串行化调度的例子

- [例11.2]现在有两个事务，分别包含下列操作：
 - 事务 T_1 ：读B； $A=B+1$ ； 写回A
 - 事务 T_2 ：读A； $B=A+1$ ； 写回B
 - 现给出对这两个事务不同的调度策略



串行调度,正确的调度

T_1	T_2
Slock B	
$Y=R(B)=2$	
Unlock B	
Xlock A	
$A=Y+1=3$	
$W(A)$	
Unlock A	
	Slock A
	$X=R(A)=3$
	Unlock A
	Xlock B
	$B=X+1=4$
	$W(B)$
	Unlock B

- 假设A、B的初值均为2。
- 按 $T_1 \rightarrow T_2$ 次序执行结果为 $A=3, B=4$
- 串行调度策略,正确的调度

串行调度(a)



串行调度,正确的调度

T_1	T_2
	Slock A
	$X=R(A)=2$
	Unlock A
	Xlock B
	$B=X+1=3$
	$W(B)$
	Unlock B
Slock B	
$Y=R(B)=3$	
Unlock B	
Xlock A	
$A=Y+1=4$	
$W(A)$	
Unlock A	

串行调度(b)

- 假设A、B的初值均为2。
- $T_2 \rightarrow T_1$ 次序执行结果为 $B=3, A=4$
- 串行调度策略,正确的调度



不可串行化调度，错误的调度

T_1	T_2
Slock B	
$Y=R(B)=2$	
	Slock A
	$X=R(A)=2$
Unlock B	
	Unlock A
Xlock A	
$A=Y+1=3$	
$W(A)$	
	Xlock B
	$B=X+1=3$
	$W(B)$
Unlock A	
	Unlock B

- 执行结果与(a)、(b)的结果都不同
- 是错误的调度

不可串行化的调度



可串行化调度，正确的调度

T_1	T_2
Slock B	
$Y=R(B)=2$	
Unlock B	
Xlock A	
	Slock A
$A=Y+1=3$	等待
$W(A)$	等待
Unlock A	等待
	$X=R(A)=3$
	Unlock A
	Xlock B
	$B=X+1=4$
	$W(B)$
	Unlock B

可串行化的调度

- 执行结果与串行调度(a)的执行结果相同
- 是正确的调度



冲突可串行化 1

- 冲突可串行化
 - 一个比可串行化更严格的条件
 - 商用系统中的调度器采用
- 冲突操作：是指不同的事务对同一数据的读写操作和写写操作：
 - $R_i(x)$ 与 $W_j(x)$ /*事务 T_i 读 x , T_j 写 x , 其中 $i \neq j$ */
 - $W_i(x)$ 与 $W_j(x)$ /*事务 T_i 写 x , T_j 写 x , 其中 $i \neq j$ */
- 其他操作是不冲突操作
- 不能交换 (Swap) 的动作：
 - 同一事务的两个操作
 - 不同事务的冲突操作



冲突可串行化 2

- 一个调度 Sc 在保证冲突操作的次序不变的情况下，通过交换两个事务不冲突操作的次序得到另一个调度 Sc' ，如果 Sc' 是串行的，称调度 Sc 是冲突可串行化的调度
- 若一个调度是冲突可串行化，则一定是可串行化的调度
- 可用这种方法判断一个调度是否是冲突可串行化的



冲突可串行化 3

- [例11.3] 今有调度

$$Sc_1 = r_1(A)w_1(A)r_2(A)w_2(A)r_1(B)w_1(B)r_2(B)w_2(B)$$

$$Sc_2 = r_1(A)w_1(A)r_1(B)w_1(B)r_2(A)w_2(A)r_2(B)w_2(B)$$

$T_1 \qquad \qquad \qquad T_2$

- Sc_2 等价于一个串行调度 T_1, T_2 。所以 Sc_1 冲突可串行化的调度



冲突可串行化调度

- 冲突可串行化调度是可串行化调度的充分条件，不是必要条件。还有不满足冲突可串行化条件的可串行化调度。
- [例11.4]有3个事务， $T_1=W_1(Y)W_1(X)$ ， $T_2=W_2(Y)W_2(X)$ ， $T_3=W_3(X)$
 - 调度 $L_1=W_1(Y)W_1(X)W_2(Y)W_2(X)W_3(X)$ 是一个串行调度。
 - 调度 $L_2=W_1(Y)W_2(Y)W_2(X)W_1(X)W_3(X)$ 不满足冲突可串行化。但是调度 L_2 是可串行化的，因为 L_2 执行的结果与调度 L_1 相同， Y 的值都等于 T_2 的值， X 的值都等于 T_3 的值



两段锁协议 1

- 数据库管理系统普遍采用两段锁协议的方法实现并发调度的可串行性，从而保证调度的正确性
- 两段锁协议，指所有事务必须分两个阶段对数据项加锁和解锁
 - 在对任何数据进行读、写操作之前，事务首先要获得对该数据的封锁
 - 在释放一个封锁之后，事务不再申请和获得任何其他封锁



两段锁协议 2

- “两段”锁的含义，事务分为两个阶段
 - 第一阶段是获得封锁，也称为扩展阶段
 - ◆ 事务可以申请获得任何数据项上的任何类型的锁，但是不能释放任何锁
 - 第二阶段是释放封锁，也称为收缩阶段
 - ◆ 事务可以释放任何数据项上的任何类型的锁，但是不能再申请任何锁
- 例

- 事务 T_i 遵守两段锁协议，其封锁序列是：

Slock A Slock B Xlock C Unlock B Unlock A Unlock C;

|← 扩展阶段 →| |← 收缩阶段 →|

- 事务 T_j 不遵守两段锁协议，其封锁序列是：

Slock A Unlock A Slock B Xlock C Unlock C Unlock B;



两段锁协议 3

事务T ₁	事务T ₂
Slock A	
R(A)=260	
	Slock C
	R(C)=300
Xlock A	
W(A)=160	
	Xlock C
	W(C)=250
	Slock A
Slock B	等待
R(B)=1000	等待
Xlock B	等待
W(B)=1100	等待
Unlock A	等待
	R(A)=160
	Xlock A
Unlock B	
	W(A)=210
	Unlock C

- 左图的调度是遵守两段锁协议的，因此一定是一个可串行化调度。
- 如何验证？

遵守两段锁协议的可串行化调度



两段锁协议 4

- 事务遵守两段锁协议是可串行化调度的充分条件，而不是必要条件。
- 若并发事务都遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的
- 若并发事务的一个调度是可串行化的，不一定所有事务都符合两段锁协议



两段锁协议 5

- 两段锁协议与防止死锁的一次封锁法
 - 一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此一次封锁法遵守两段锁协议
 - 但是两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁



两段锁协议 6

● [例]遵守两段锁协议的事务发生死锁

事务 T_1	事务 T_2
Slock B	
R(B)=2	
	Slock A
	R(A)=2
Xlock A	
等待	Xlock A
等待	等待

遵守两段锁协议的事务可能发生死锁





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch57 封锁粒度

Software Institute
Nanjing University
Bei Jia

封锁粒度

- 封锁对象的大小称为封锁粒度(Granularity)
- 封锁的对象：逻辑单元，物理单元
- 例：在关系数据库中，封锁对象：
 - 逻辑单元：属性值、属性值的集合、元组、关系、索引项、整个索引、整个数据库等
 - 物理单元：页（数据页或索引页）、物理记录等



选择封锁粒度原则

- 封锁粒度与系统的并发度和并发控制的开销密切相关。
 - 封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；
 - 封锁的粒度越小，并发度较高，但系统开销也就越大
- 例
 - 若封锁粒度是数据页，事务 T_1 需要修改元组 L_1 ，则 T_1 必须对包含 L_1 的整个数据页 A 加锁。如果 T_1 对 A 加锁后事务 T_2 要修改 A 中元组 L_2 ，则 T_2 被迫等待，直到 T_1 释放 A 。
 - 如果封锁粒度是元组，则 T_1 和 T_2 可以同时 L_1 和 L_2 加锁，不需要互相等待，提高了系统的并行度。
 - 又如，事务 T 需要读取整个表，若封锁粒度是元组， T 必须对表中的每一个元组加锁，开销极大



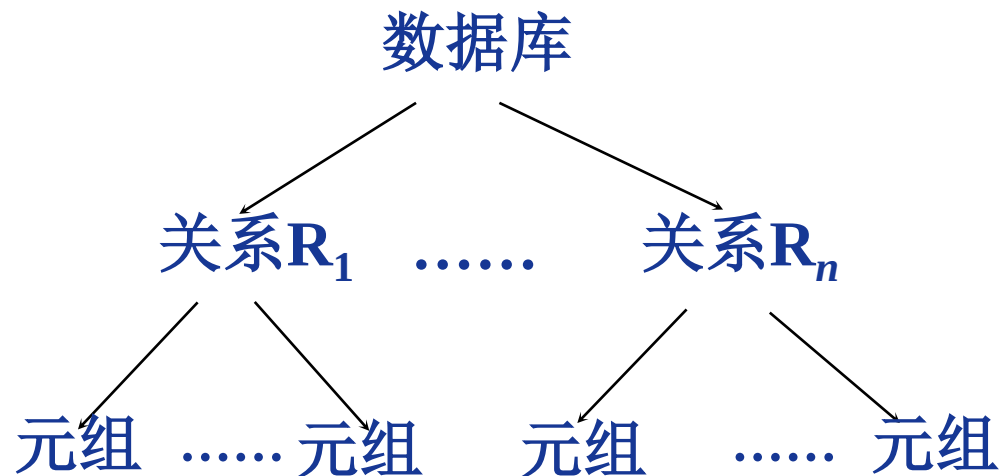
多粒度封锁

- 多粒度封锁(Multiple Granularity Locking)
 - 在一个系统中同时支持多种封锁粒度供不同的事务选择
- 选择封锁粒度, 同时考虑封锁开销和并发度两个因素, 适当选择封锁粒度
 - 需要处理多个关系的大量元组的用户事务: 以数据库为封锁单位
 - 需要处理大量元组的用户事务: 以关系为封锁单元
 - 只处理少量元组的用户事务: 以元组为封锁单位



多粒度树

- 多粒度树
 - 以树形结构来表示多级封锁粒度
 - 根节点是整个数据库，表示最大的数据粒度
 - 叶结点表示最小的数据粒度
- 例：三级粒度树。根结点为数据库，数据库的子结点为关系，关系的子结点为元组。



多粒度封锁协议

- 允许多粒度树中的每个结点被独立地加锁
- 对一个结点加锁意味着这个结点的所有后裔结点也被加以同样类型的锁
- 在多粒度封锁中一个数据对象可能以两种方式封锁：
 - 显式封锁: 直接加到数据对象上的封锁
 - 隐式封锁: 是该数据对象没有独立加锁, 是由于其上级结点加锁而使该数据对象加上了锁
- 显式封锁和隐式封锁的效果是一样的



显式封锁和隐式封锁 1

- 系统检查封锁冲突时
 - 要检查显式封锁
 - 还要检查隐式封锁
- 例如，事务T要对关系 R_1 加X锁
 - 系统必须搜索其上级结点数据库、关系 R_1
 - 还要搜索 R_1 的下级结点，即 R_1 中的每一个元组
 - 如果其中某一个数据对象已经加了不相容锁，则T必须等待



显式封锁和隐式封锁 2

- 对某个数据对象加锁，系统要检查
 - 该数据对象
 - ◆ 有无显式封锁与之冲突
 - 所有上级结点
 - ◆ 检查本事务的显式封锁是否与该数据对象上的隐式封锁冲突：(由上级结点已加的封锁造成的)
 - 所有下级结点
 - ◆ 看上面的显式封锁是否与本事务的隐式封锁（将加到下级结点的封锁）冲突



意向锁

- 引进意向锁 (intention lock) 目的
 - 提高对某个数据对象加锁时系统的检查效率
- 如果对一个结点加意向锁, 则说明该结点的下层结点正在被加锁
- 对任一结点加基本锁, 必须先对它的上层结点加意向锁
- 例如, 对任一元组加锁时, 必须先对它所在的数据库和关系加意向锁



常用意向锁-意向共享锁

- 意向共享锁(Intent Share Lock, 简称IS锁)
 - 如果对一个数据对象加IS锁, 表示它的后裔结点拟 (意向) 加S锁。
 - 例如: 事务 T_1 要对 R_1 中某个元组加S锁, 则要首先对关系 R_1 和数据库加IS锁



常用意向锁-意向排它锁

- 意向排它锁(Intent Exclusive Lock, 简称IX锁)
 - 如果对一个数据对象加IX锁, 表示它的后裔结点拟 (意向) 加X锁。
 - 例如: 事务 T_1 要对 R_1 中某个元组加X锁, 则要首先对关系 R_1 和数据库加IX锁



常用意向锁-共享意向排它锁

- 共享意向排它锁(Share Intent Exclusive Lock, 简称SIX锁)
 - 如果对一个数据对象加SIX锁, 表示对它加S锁, 再加IX锁, 即 $SIX = S + IX$ 。
- 例: 对某个表加SIX锁, 则表示该事务要读整个表 (所以要对该表加S锁), 同时会更新个别元组 (所以要对该表加IX锁)。



意向锁的相容矩阵

$T_1 \backslash T_2$	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

Y=Yes, 表示相容的请求

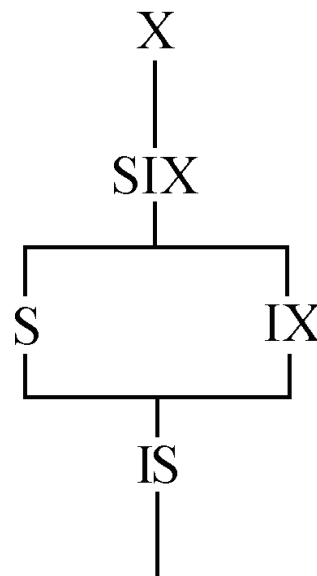
N=No, 表示不相容的请求

(a) 数据锁的相容矩阵



锁的强度

- 锁的强度是指它对其他锁的排斥程度
- 一个事务在申请封锁时以强锁代替弱锁是安全的，反之则不然



(b) 锁的强度的偏序关系

具有意向锁的多粒度封锁方法

- 申请封锁时应该按自上而下的次序进行
- 释放封锁时则应该按自下而上的次序进行
- 具有意向锁的多粒度封锁方法
 - 提高了系统的并发度
 - 减少了加锁和解锁的开销
 - 在实际的数据库管理系统产品中得到广泛应用
- 例如：事务 T_1 要对关系 R_1 加S锁
 - 要首先对数据库加IS锁
 - 检查数据库和 R_1 是否已加了不相容的锁(X或IX)
 - 不再需要搜索和检查 R_1 中的元组是否加了不相容的锁(X锁)





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE

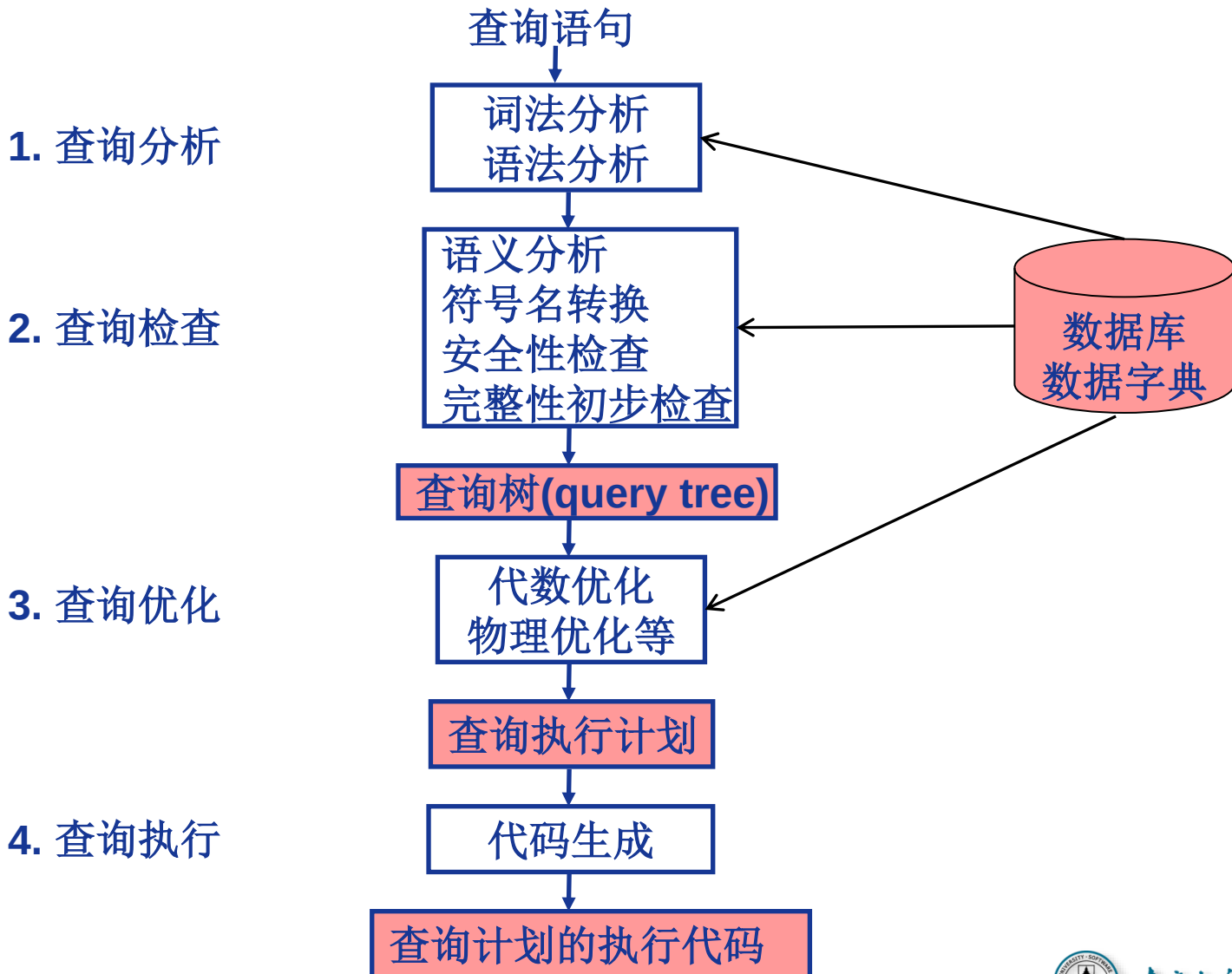


数据管理基础

ch57 查询处理

Software Institute
Nanjing University
Bei Jia

查询处理步骤



查询分析

- 查询分析的任务：对查询语句进行扫描、词法分析和语法分析
 - 词法分析：从查询语句中识别出正确的语言符号
 - 语法分析：进行语法检查



查询检查 1

- 查询检查的任务
 - 合法权检查
 - 视图转换
 - 安全性检查
 - 完整性初步检查
- 根据数据字典中有关的模式定义检查语句中的数据库对象，如关系名、属性名是否存在和有效
- 如果是对视图的操作，则要用视图消解方法把对视图的操作转换成对基本表的操作



查询检查 2

- 根据数据字典中的用户权限和完整性约束定义对用户的存取权限进行检查
- 检查通过后把SQL查询语句转换成内部表示，即等价的关系代数表达式。
- 关系数据库管理系统一般都用查询树，也称为语法分析树来表示扩展的关系代数表达式。



查询优化

- 查询优化：选择一个高效执行的查询处理策略
- 查询优化分类
 - 代数优化/逻辑优化：指关系代数表达式的优化
 - 物理优化：指存取路径和底层操作算法的选择
- 查询优化的选择依据
 - 基于规则(rule based)
 - 基于代价(cost based)
 - 基于语义(semantic based)



查询执行

- 依据优化器得到的执行策略生成查询执行计划
- 代码生成器(code generator)生成执行查询计划的代码
- 两种执行方法
 - 自顶向下
 - 自底向上



选择操作的实现 1

- 选择操作典型实现方法：

- 全表扫描方法 (Table Scan)

- ◆ 对查询的基本表顺序扫描，逐一检查每个元组是否满足选择条件，把满足条件的元组作为结果输出
 - ◆ 适合小表，不适合大表

- 索引扫描方法 (Index Scan)

- ◆ 适合于选择条件中的属性上有索引(例如B+树索引或Hash索引)
 - ◆ 通过索引先找到满足条件的元组主码或元组指针，再通过元组指针直接在查询的基本表中找到元组



选择操作的实现 2

- [例9.1] SELECT *

FROM Student

WHERE <条件表达式>

- 考虑<条件表达式>的几种情况:

C1: 无条件;

C2: Sno = '201215121';

C3: Sage>20;

C4: Sdept = 'CS' AND Sage>20;



选择操作的实现 3

● 全表扫描算法

■ 假设可以使用的内存为M块，全表扫描算法思想：

- ① 按照物理次序读Student的M块到内存
- ② 检查内存的每个元组t，如果满足选择条件，则输出t
- ③ 如果student还有其他块未被处理，重复①和②



选择操作的实现 4

- 索引扫描算法
- [例9.1-C2] SELECT *

FROM Student

WHERE Sno='201215121'

- 假设Sno上有索引(或Sno是散列码)
- 算法：
 - ◆ 使用索引(或散列)得到Sno为‘201215121’元组的指针
 - ◆ 通过元组指针在Student表中检索到该学生



选择操作的实现 5

- [例9.1-C3] SELECT *

FROM Student

WHERE Sage>20

- 假设Sage 上有B+树索引

- 算法：

- ◆ 使用B+树索引找到Sage=20的索引项，以此为入口点在B+树的顺序集上得到Sage>20的所有元组指针
- ◆ 通过这些元组指针到student表中检索到所有年龄大于20的学生。



选择操作的实现 6

- [例9.1-C4] SELECT *

FROM Student

WHERE Sdept='CS' AND Sage>20;

- 假设Sdept和Sage上都有索引

- 算法一：分别用Index Scan找到Sdept = 'CS'的一组元组指针和Sage>20的另一组元组指针

- ◆ 求这两组指针的交集
- ◆ 到Student表中检索
- ◆ 得到计算机系年龄大于20的学生

- 算法二：找到Sdept='CS'的一组元组指针，

- ◆ 通过这些元组指针到Student表中检索
- ◆ 并对得到的元组检查另一些选择条件(如Sage>20)是否满足
- ◆ 把满足条件的元组作为结果输出。



连接操作的实现

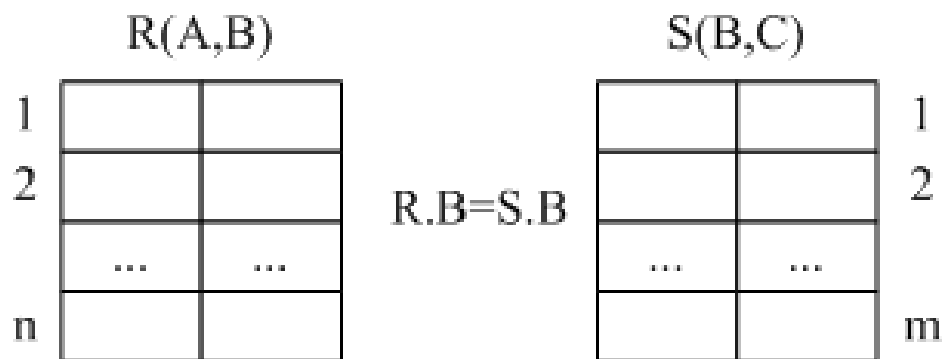
- 连接操作是查询处理中最耗时的操作之一
- 本节只讨论等值连接(或自然连接)最常用的实现算法
 - 嵌套循环算法(nested loop join)
 - 排序-合并算法(sort-merge join 或merge join)
 - 索引连接(index join)算法
 - Hash Join算法
- [例9.2]

```
SELECT *  
FROM   Student, SC  
WHERE  Student.Sno=SC.Sno;
```



嵌套循环算法

- 嵌套循环算法(nested loop join)
 - 对外层循环(Student表)的每一个元组(s), 检索内层循环(SC表)中的每一个元组(sc)
 - 检查这两个元组在连接属性(Sno)上是否相等
 - 如果满足连接条件, 则串接后作为结果输出, 直到外层循环表中的元组处理完为止。



排序-合并算法 1

- 排序-合并算法(sort-merge join 或merge join)
 - ① 如果连接的表没有排好序, 先对Student表和SC表按连接属性Sno排序
 - ② 取Student表中第一个Sno, 依次扫描SC表中具有相同Sno的元组
 - ③ 当扫描到Sno不相同的第一个SC元组时, 返回Student表扫描它的下一个元组, 再扫描SC表中具有相同Sno的元组, 把它们连接起来
 - 重复上述步骤直到Student 表扫描完
- Student表和SC表都只要扫描一遍
- 如果两个表原来无序, 执行时间要加上对两个表的排序时间
- 对于大表, 先排序后使用排序-合并连接算法执行连接, 总的时间一般仍会减少



排序-合并算法 2

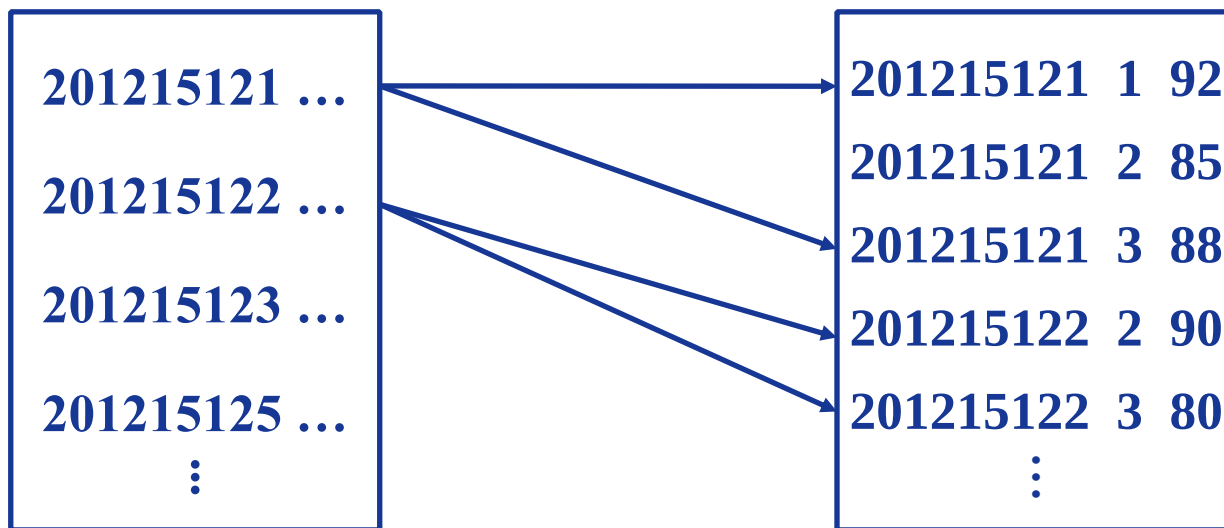
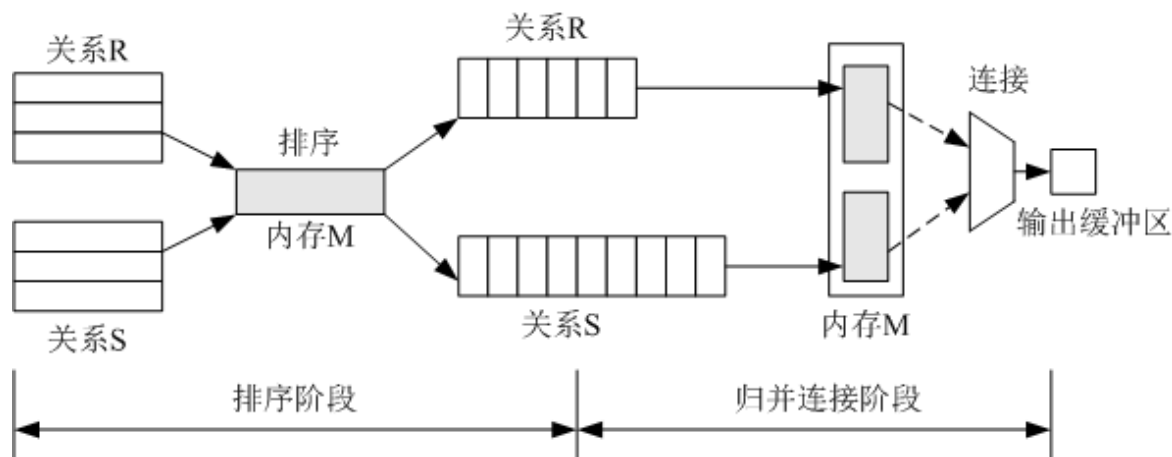


图9.2 排序-合并连接方法示意图

索引连接算法

● 索引连接(index join)算法

■ 步骤:

- ① 在SC表上已经建立属性Sno的索引。
- ② 对Student中每一个元组，由Sno值通过SC的索引查找相应的SC元组。
- ③ 把这些SC元组和Student元组连接起来

■ 循环执行②③，直到Student表中的元组处理完为止



Hash Join算法

- Hash Join算法
 - 把连接属性作为hash码，用同一个hash函数把Student表和SC表中的元组散列到hash表中。
 - 划分阶段(building phase, 也称为partitioning phase)
 - ◆ 对包含较少元组的表(如Student表)进行一遍处理
 - ◆ 把它的元组按hash函数分散到hash表的桶中
 - 试探阶段(probing phase, 也称为连接阶段join phase)
 - ◆ 对另一个表(SC表)进行一遍处理
 - ◆ 把SC表的元组也按同一个hash函数（hash码是连接属性）进行散列
 - ◆ 把SC元组与桶中来自Student表并与之相匹配的元组连接起来
- hash join算法前提：假设两个表中较小的表在第一阶段后可以完全放入内存的hash桶中





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch59 查询优化

Software Institute
Nanjing University
Bei Jia

查询优化概述 1

- 关系系统的查询优化
 - 是关系数据库管理系统实现的关键技术又是关系系统的优点所在
 - 减轻了用户选择存取路径的负担
- 关系查询优化是影响关系数据库管理系统性能的关键因素
- 由于关系表达式的语义级别很高，使关系系统可以从关系表达式中分析查询语义，提供了执行查询优化的可能性



查询优化概述 2

● 非关系系统

- 用户使用过程化的语言表达查询要求，执行何种记录级的操作，以及操作的序列是由用户来决定的
- 用户必须了解存取路径，系统要提供用户选择存取路径的手段，查询效率由用户的存取策略决定
- 如果用户做了不当的选择，系统是无法对此加以改进的



查询优化概述 3

- 查询优化的优点

- 用户不必考虑如何最好地表达查询以获得较好的效率
- 系统可以比用户程序的“优化”做得更好
 - ◆ 优化器可以从数据字典中获取许多统计信息，而用户程序则难以获得这些信息。
 - ◆ 如果数据库的物理统计信息改变了，系统可以自动对查询重新优化以选择相适应的执行计划。在非关系系统中必须重写程序，而重写程序在实际应用中往往是不太可能的。
 - ◆ 优化器可以考虑数百种不同的执行计划，程序员一般只能考虑有限的几种可能性。
 - ◆ 优化器中包括了很多复杂的优化技术，这些优化技术往往只有最好的程序员才能掌握。系统的自动优化相当于使得所有人都拥有这些优化技术。



查询优化的总目标

- 关系数据库管理系统通过某种代价模型计算出各种查询执行策略的执行代价，然后选取代价最小的执行方案
 - 集中式数据库
 - ◆ 执行开销主要包括：磁盘存取块数(I/O代价)+处理机时间(CPU代价)+查询的内存开销
 - ◆ I/O代价是最主要的
 - 分布式数据库
 - ◆ 总代价=I/O代价+CPU代价+内存代价 + 通信代价
- 查询优化的总目标
 - 选择有效的策略
 - 求得给定关系表达式的值
 - 使得查询代价最小(实际上是较小)
- 一个关系查询可以对应不同的执行方案，其效率可能相差非常大。



查询优化的实例

- [例9.3] 求选修了2号课程的学生姓名。

- 用SQL表达：

SELECT Student.Sname

FROM Student, SC

WHERE Student.Sno=SC.Sno AND SC.Cno='2'

- 假定学生-课程数据库中有1000个学生记录，10000个选课记录；选修2号课程的选课记录为50个

- 可以用多种等价的关系代数表达式来完成这一查询

- ◆ $Q_1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$

- ◆ $Q_2 = \pi_{Sname}(\sigma_{SC.Cno='2'}(Student \bowtie SC))$

- ◆ $Q_3 = \pi_{Sname}(Student \bowtie \sigma_{SC.Cno='2'}(SC))$



方案A 1

● $Q_1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$

1. 计算广义笛卡尔积

- 在内存中尽可能多地装入某个表(如 Student表)的若干块, 留出一块存放另一个表(如SC表)的元组。
- 把SC中的每个元组和Student中每个元组连接, 连接后的元组装满一块后就写到中间文件上
- 从SC中读入一块和内存中的Student元组连接, 直到SC表处理完。
- 再读入若干块Student元组, 读入一块SC元组
- 重复上述处理过程, 直到把Student表处理完
- 设一个块能装10个Student元组或100个SC元组, 在内存中存放5块Student元组和1块SC元组, 则读取总块数为 $\frac{1000}{10} + \frac{1000}{10 \times 5} \times \frac{10000}{100} = 100 + 20 \times 100 = 2100$ 块
- 读Student表100块, 读SC表20遍, 每遍100块, 则总计要读取2100数据块。
- 连接后的元组数为 $10^3 \times 10^4 = 10^7$ 。设每块能装10个元组, 则写出 10^6 块。



方案A 2

2. 作选择操作

- 依次读入连接后的元组，按照选择条件选取满足要求的记录
- 假定内存处理时间忽略。读取中间文件花费的时间(同写中间文件一样)需读入 10^6 块。
- 若满足条件的元组假设仅50个，均可放在内存。

3. 作投影操作

- 把第2步的结果在Sname上作投影输出，得到最终结果
- 方案1执行查询的总读写数据块= $2100 + 10^6 + 10^6$



方案B

● $Q_2 = \pi_{Sname}(\sigma_{Sc.Cno='2'}(Student \bowtie SC))$

1. 计算自然连接

- 执行自然连接，读取Student和SC表的策略不变，总的读取块数仍为2100块
- 自然连接的结果比第一种情况大大减少，为 10^4 个元组
- 写出数据块 = 10^3 块

2. 读取中间文件块，执行选择运算，读取的数据块 = 10^3 块

3. 把第2步结果投影输出。

- 方案B执行查询的总读写数据块 = $2100 + 10^3 + 10^3$
- 其执行代价大约是方案A的488分之一



方案C

● $Q_3 = \pi_{Sname}(Student \bowtie \sigma_{SC.Cno='2'}(SC))$

1. 先对SC表作选择运算，只需读一遍SC表，存取100块，因为满足条件的元组仅50个，不必使用中间文件。
2. 读取Student表，把读入的Student元组和内存中的SC元组作连接。也只需读一遍Student表共100块。
3. 把连接结果投影输出

■ 方案C总的读写数据块=100+100

■ 其执行代价大约是方案A的万分之一，是方案B的20分之一



拥有索引的实例

- 假如SC表的Cno字段上有索引
 - 第一步就不必读取所有的SC元组而只需读取Cno='2'的那些元组(50个)
 - 存取的索引块和SC中满足条件的数据块大约总共3 ~ 4块
- 若Student表在Sno上也有索引
 - 不必读取所有的Student元组
 - 因为满足条件的SC记录仅50个，涉及最多50个Student记录
 - 读取Student表的块数也可大大减少



优化实例

- 把代数表达式Q1变换为Q2、 Q3

$$Q_1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge Sc.Cno='2'}(Student \times SC))$$



$$Q_2 = \pi_{Sname}(\sigma_{Sc.Cno='2'}(Student \bowtie SC))$$

$$Q_3 = \pi_{Sname}(Student \bowtie \sigma_{SC.Cno='2'}(SC))$$

- 有选择和连接操作时，先做选择操作，这样参加连接的元组就可以大大减少，这是代数优化
- 在Q₃中
 - SC表的选择操作算法有全表扫描或索引扫描，经过初步估算，索引扫描方法较优。
 - 对于Student和SC表的连接，利用Student表上的索引，采用索引连接代价也较小，这就是物理优化。



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch60 代数优化

Software Institute
Nanjing University
Bei Jia

关系代数表达式等价变换规则

- 代数优化策略：通过对关系代数表达式的等价变换来提高查询效率
- 关系代数表达式的等价：指用相同的关系代替两个表达式中相应的关系所得到的结果是相同的
- 两个关系表达式 E_1 和 E_2 是等价的，可记为 $E_1 \equiv E_2$



常用的等价变换规则 1

1. 连接、笛卡尔积交换律

设 E_1 和 E_2 是关系代数表达式, F 是连接运算的条件, 则有

$$E_1 \times E_2 \equiv E_2 \times E_1$$

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

$$E_1 \underset{F}{\bowtie} E_2 \equiv E_2 \underset{F}{\bowtie} E_1$$

2. 连接、笛卡尔积的结合律

设 E_1, E_2, E_3 是关系代数表达式, F_1 和 F_2 是连接运算的条件

$$(E_1 \times E_2) \times E_3 \equiv E_1 \times (E_2 \times E_3)$$

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \underset{F_1}{\bowtie} E_2) \underset{F_2}{\bowtie} E_3 \equiv E_1 \underset{F_2}{\bowtie} (E_2 \underset{F_2}{\bowtie} E_3)$$

3. 投影的串接定律

$$\pi_{A_1, A_2, \dots, A_n} (\pi_{B_1, B_2, \dots, B_m} (E)) \equiv \pi_{A_1, A_2, \dots, A_n} (E)$$

- E 是关系代数表达式
- $A_i (i=1, 2, \dots, n), B_j (j=1, 2, \dots, m)$ 是属性名
- $\{A_1, A_2, \dots, A_n\}$ 构成 $\{B_1, B_2, \dots, B_m\}$ 的子集

4. 选择的串接定律

$$\sigma_{F_1} (\sigma_{F_2} (E)) \equiv \sigma_{F_1 \wedge F_2} (E)$$

- E 是关系代数表达式, F_1, F_2 是选择条件
- 选择的串接律说明选择条件可以合并, 这样一次就可检查全部条件



常用的等价变换规则 2

5. 选择与投影操作的交换律

$$\sigma_F(\pi_{A_1, A_2, \dots, A_n}(E)) \equiv \pi_{A_1, A_2, \dots, A_n}(\sigma_F(E))$$

- 选择条件F只涉及属性 $A_1, \dots, A_n$ 。
- 若F中不属于 $A_1, \dots, A_n$ 的属性 $B_1, \dots, B_m$ 有更一般规则:

$$\pi_{A_1, A_2, \dots, A_n}(\sigma_F(E)) \equiv \pi_{A_1, A_2, \dots, A_n}(\sigma_F(\pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E)))$$

6. 选择与笛卡尔积的交换律

- 如果F中涉及的属性都是 E_1 中的属性, 则 $\sigma_F(E_1 \times E_2) \equiv \sigma_F(E_1) \times E_2$
- 如果 $F = F_1 \wedge F_2$, 并且 F_1 只涉及 E_1 中的属性, F_2 只涉及 E_2 中的属性, 则由上面的等价变换规则1, 4, 6可推出: $\sigma_F(E_1 \times E_2) \equiv \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$
- 若 F_1 只涉及 E_1 中的属性, F_2 涉及 E_1 和 E_2 两者的属性, 则仍有
$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_2}(\sigma_{F_1}(E_1) \times E_2)$$
- 它使部分选择在笛卡尔积前先做。

常用的等价变换规则 3

7. 选择与并的分配律

设 $E=E_1 \cup E_2$, E_1, E_2 有相同的属性名, 则

$$\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$$

8. 选择与差运算的分配律

若 E_1 与 E_2 有相同的属性名, 则

$$\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$$

9. 选择对自然连接的分配律

$$\sigma_F(E_1 \bowtie E_2) \equiv \sigma_F(E_1) \bowtie \sigma_F(E_2)$$

F 只涉及 E_1 与 E_2 的公共属性

常用的等价变换规则 4

10. 投影与笛卡尔积的分配律

设 E_1 和 E_2 是两个关系表达式, $A_1, \dots, A_n$ 是 E_1 的属性, $B_1, \dots, B_m$ 是 E_2 的属性, 则

$$\pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m} (E_1 \times E_2) \equiv \pi_{A_1, A_2, \dots, A_n} (E_1) \times \pi_{B_1, B_2, \dots, B_m} (E_2)$$

11. 投影与并的分配律

设 E_1 和 E_2 有相同的属性名, 则

$$\pi_{A_1, A_2, \dots, A_n} (E_1 \cup E_2) \equiv \pi_{A_1, A_2, \dots, A_n} (E_1) \cup \pi_{A_1, A_2, \dots, A_n} (E_2)$$

典型的启发式规则

1. 选择运算应尽可能先做
 - 在优化策略中这是最重要、最基本的一条。
2. 把投影运算和选择运算同时进行
 - 如有若干投影和选择运算，并且它们都对同一个关系操作，则可以在扫描此关系的同时完成所有的这些运算以避免重复扫描关系。
3. 把投影同其前或其后的双目运算结合起来
 - 没有必要为了去掉某些字段而扫描一遍关系。
4. 把某些选择同在它前面要执行的笛卡尔积结合起来成为一个连接运算
 - 连接特别是等值连接运算要比同样关系上的笛卡尔积省很多时间。
5. 找出公共子表达式
 - 如果这种重复出现的子表达式的结果不是很大的关系，并且从外存中读入这个关系比计算该子表达式的时间少得多，则先计算一次公共子表达式并把结果写入中间文件是合算的。
 - 当查询的是视图时，定义视图的表达式就是公共子表达式的情况



查询树的启发式优化 1

- 遵循这些启发式规则，应用等价变换公式来优化关系表达式的算法。

- 算法：关系表达式的优化
- 输入：一个关系表达式的查询树
- 输出：优化的查询树
- 方法：

规则4：选择的串接定律

$$\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_1 \wedge F_2}(E)$$

- ① 利用等价变换规则4把形如 $\sigma_{F_1 \wedge F_2 \wedge \dots \wedge F_n}(E)$ 变换为 $\sigma_{F_1}(\sigma_{F_2}(\dots(\sigma_{F_n}(E))\dots))$ 。
- ② 对每一个选择，利用等价变换规则4 ~ 9尽可能把它移到树的叶端。

规则4： 合并或分解选择运算

规则5-9： 选择运算与其他运算交换

查询树的启发式优化 2

- ③ 对每一个投影利用等价变换规则3, 5, 10, 11中的一般形式
尽可能把它

– 注意:

规则3: 合并或分解投影运算
规则5,10,11: 投影运算与其他运算交换

- ✓ 等价变换规则3使一些投影消失或使一些投影出现
- ✓ 规则5把一个投影分裂为两个, 其中一个有可能被移向树的叶端

- ④ 利用等价变换规则3 ~ 5, 把选择和投影的串接合并成单个选择、单个投影或一个选择后跟一个投影, 使多个选择或投影能同时执行

规则3: 合并或分解投影运算
规则4: 合并或分解选择运算
规则5: 投影运算与选择运算交换

查询树的启发式优化 3

- ⑤ 把上述得到的语法树的内节点分组。
- 每一双目运算($\times$, $\bowtie$, $\cup$, $-$)和它所有的直接祖先为一组(这些直接祖先是(σ , π 运算)。
 - 如果其后代直到叶子全是单目运算, 则也将它们并入该组
 - 但当双目运算是笛卡尔积($\times$), 而且后面不是与它组成等值连接的选择时, 则不能把选择与这个双目运算组成同一组



一个例子 1

- [例9.4]下面给出[例9.3]中 SQL语句的代数优化示例

① 把SQL语句转换成查询树，如下图所示：

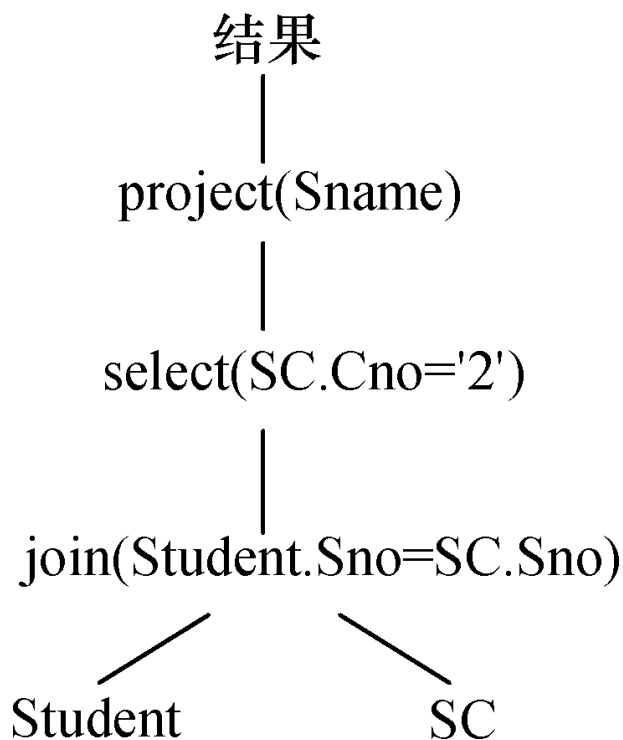


图9.3 查询树图

一个例子 2

- 为了使用关系代数表达式的优化法，假设内部表示是关系代数语法树，则上面的查询树如图9.4所示。

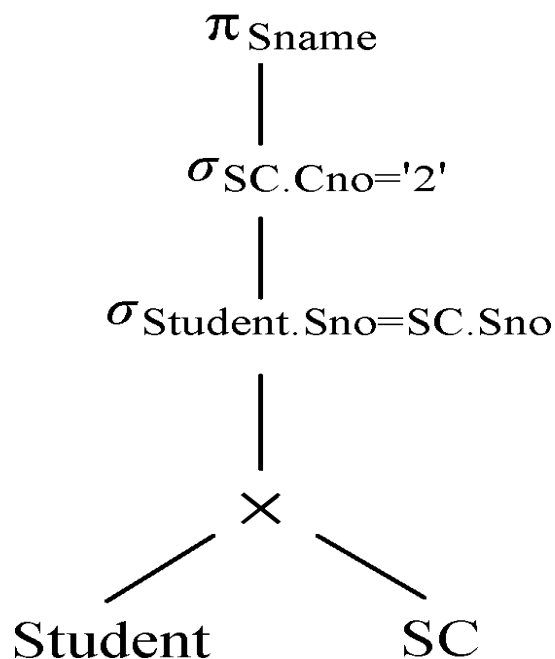
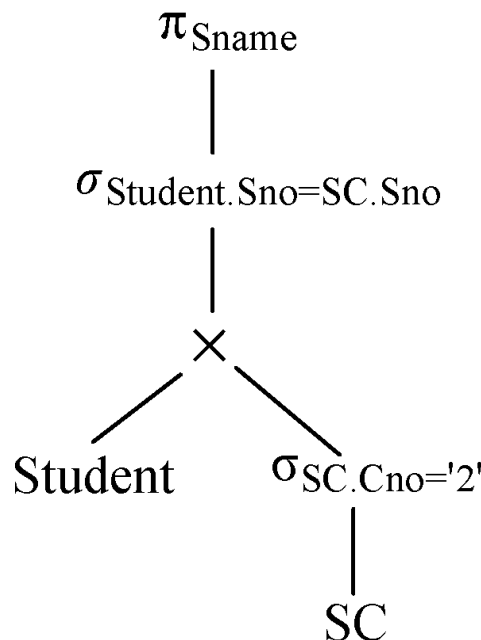


图9.4 关系代数语法树图

一个例子 3

② 对查询树进行优化

- ◆ 利用规则4、6把选择 $\sigma_{SC.Cno='2'}$ 移到叶端，图9.4查询树便转换成下图优化的查询树。这就是9.2.2节中Q3的查询树表示。





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch61 物理优化

Software Institute
Nanjing University
Bei Jia

物理优化

- 代数优化改变查询语句中操作的次序和组合，不涉及底层的存取路径
- 对于一个查询语句有许多存取方案，它们的执行效率不同，仅仅进行代数优化是不够的
- 物理优化就是要选择高效合理的操作算法或存取路径，求得优化的查询计划
- 物理优化方法
 - 基于规则的启发式优化
 - ◆ 启发式规则是指那些在大多数情况下都适用，但在不是每种情况下都是最好的规则。
 - 基于代价估算的优化
 - ◆ 优化器估算不同执行策略的代价，并选出具有最小代价的执行计划。
 - 两者结合的优化方法：
 - ◆ 常常先使用启发式规则，选取若干较优的候选方案，减少代价估算的工作量
 - ◆ 然后分别计算这些候选方案的执行代价，较快地选出最终的优化方案



选择操作的启发式规则

- 对于小关系，使用全表顺序扫描，即使选择列上有索引
- 对于大关系，启发式规则有：
 1. 对于选择条件是“主码 = 值”的查询
 - 查询结果最多是一个元组，可以选择主码索引
 - 一般的关系数据库管理系统会自动建立主码索引
 2. 对于选择条件是“非主属性 = 值”的查询，并且选择列上有索引
 - 要估算查询结果的元组数目
 - ◆ 如果比例较小(<10%)可以使用索引扫描方法
 - ◆ 否则还是使用全表顺序扫描
 3. 对于选择条件是属性上的非等值查询或者范围查询，并且选择列上有索引
 - 要估算查询结果的元组数目
 - ◆ 如果比例较小(<10%)可以使用索引扫描方法
 - ◆ 否则还是使用全表顺序扫描
 4. 对于用AND连接的合取选择条件
 - 如果有涉及这些属性的组合索引
 - ◆ 优先采用组合索引扫描方法
 - 如果某些属性上有一般的索引，可以用索引扫描方法
 - ◆ 通过分别查找满足每个条件的指针，求指针的交集
 - ◆ 通过索引查找满足部分条件的元组，然后在扫描这些元组时判断是否满足剩余条件
 - 其他情况：使用全表顺序扫描
 5. 对于用OR连接的析取选择条件，一般使用全表顺序扫描



连接操作的启发式规则

1. 如果2个表都已经按照连接属性排序
 - 选用排序-合并算法
2. 如果一个表在连接属性上有索引
 - 选用索引连接算法
3. 如果上面2个规则都不适用，其中一个表较小
 - 选用hash join算法
4. 可以选用嵌套循环方法，并选择其中较小的表，确切地讲是占用的块数(B)较少的表，作为外表(外循环的表)。
 - 理由：
 - ◆ 设连接表R与S分别占用的块数为 B_r 与 B_s ，连接操作使用的内存缓冲区块数为 K ，分配 $K-1$ 块给外表，如果R为外表，则嵌套循环法存取的块数为 $B_r + B_r B_s / (K-1)$
 - ◆ 显然应该选块数小的表作为外表



基于代价的优化

- 启发式规则优化是定性的选择，适合解释执行的系统
 - 解释执行的系统，优化开销包含在查询总开销之中
- 编译执行的系统中查询优化和查询执行是分开的
 - 可以采用精细复杂一些的基于代价的优化方法



统计信息

- 基于代价的优化方法要计算查询的各种不同执行方案的执行代价，它与数据库的状态密切相关

- 优化器需要的统计信息

- 对每个基本表

- ◆ 该表的元组总数(N)
- ◆ 元组长度(l)
- ◆ 占用的块数(B)
- ◆ 占用的溢出块数(BO)

- 对索引

- ◆ 索引的层数(L)
- ◆ 不同索引值的个数
- ◆ 索引的选择基数S(有S个元组具有某个索引值)
- ◆ 索引的叶结点数(Y)

- 对基表的每个列

- ◆ 该列不同值的个数(m)
- ◆ 列最大值
- ◆ 列最小值
- ◆ 列上是否已经建立了索引
- ◆ 哪种索引(B+树索引、Hash索引、聚集索引)
- ◆ 可以计算选择率(f)
 - 如果不同值的分布是均匀的, $f = 1/m$
 - 如果不同值的分布不均匀, 则要计算每个值的选择率, $f = \text{具有该值的元组数}/N$



代价估算 1

- 全表扫描算法的代价估算公式

- 如果基本表大小为B块，全表扫描算法的代价 $\text{cost} = B$
- 如果选择条件是“码 = 值”，那么平均搜索代价 $\text{cost} = B/2$

- 索引扫描算法的代价估算公式

- 如果选择条件是“码 = 值”
 - ◆ 则采用该表的主索引
 - ◆ 若为B+树，层数为L，需要存取B+树中从根结点到叶结点L块，再加上基本表中该元组所在的那一块，所以 $\text{cost} = L + 1$
- 如果选择条件涉及非码属性
 - ◆ 若为B+树索引，选择条件是相等比较，S是索引的选择基数(有S个元组满足条件)
 - ◆ 满足条件的元组可能会保存在不同的块上，所以(最坏的情况) $\text{cost} = L + S$
- 如果比较条件是 $>$, $> =$, $<$, $< =$ 操作
 - ◆ 假设有一半的元组满足条件
 - ◆ 就要存取一半的叶结点
 - ◆ 通过索引访问一半的表存储块
 - ◆ $\text{cost} = L + Y/2 + B/2$
 - ◆ 如果可以获得更准确的选择基数，可以进一步修正Y/2与B/2



代价估算 2

- 嵌套循环连接算法的代价估算公式

- 嵌套循环连接算法的代价 $\text{cost} = Br + BrBs/(K-1)$
- 如果需要把连接结果写回磁盘 $\text{cost} = Br + BrBs/(K-1) + (Frs * Nr * Ns) / Mrs$
 - ◆ 其中Frs为连接选择性(join selectivity), 表示连接结果元组数的比例
 - ◆ Mrs是存放连接结果的块因子, 表示每块中可以存放的结果元组数目

- 排序-合并连接算法的代价估算公式

- 如果连接表已经按照连接属性排好序, 则 $\text{cost} = Br + Bs + (Frs * Nr * Ns) / Mrs$
- 如果必须对文件排序
 - ◆ 还需要在代价函数中加上排序的代价
 - ◆ 对于包含B个块的文件排序的代价大约是 $(2*B) + (2*B*\log_2 B)$

